MINOR PROJECT 2

Methodology

Design, Development & Security Implementation

Project	Zero Trust Architecture — Web Application
Student	Himanshu Soni
Organisation	Naviotech Solutions Pvt. Ltd.
Division	Cybersecurity
Standard	NIST SP 800-207
Year	2025

1. Methodology Overview

The project follows a Security-First Development approach guided by NIST SP 800-207 Zero Trust principles. The methodology is structured in five sequential phases: Requirements Analysis, Architecture Design, Implementation, Security Testing, and Evaluation. Each phase enforces the ZTA principle of never granting implicit trust.

2. Phase 1 — Requirements Analysis

2.1 Threat Modelling

Before any code was written, a threat model was constructed to identify attack vectors relevant to a web application handling role-sensitive data. The STRIDE framework was applied:

Threat	Category	Mitigation Strategy
Spoofing identity	Spoofing	Server-side session role validation on every page load
Tampering with role in storage	Tampering	Role cannot be self-assigned; set only post-authentication
Privilege escalation via URL	Elevation of Privilege	Role check at top of each protected page
Unauthenticated direct access	Elevation of Privilege	Redirect to login if role not set
User enumeration via error messages	Information Disclosure	Generic 'Invalid Credentials' message for all failures
Brute-force login	Denial of Service	All failed attempts rejected; rate-limiting in roadmap

3. Phase 2 — Architecture Design

3.1 ZTA Design Decisions

- Single Entry Point: One login portal (login.html) for all user roles — no role selector exposed to the client.
- Server-Side Role Assignment: The role stored in localStorage is set exclusively after server-validated credentials match.
- Isolated Dashboards: admin.html and employee.html are independent environments with no shared components.
- Micro-Segmentation: Each dashboard immediately checks role on load and redirects on mismatch — effectively segmenting the application.
- Container Isolation: Docker provides network and filesystem isolation between the web server and database layers.

3.2 Access Control Flow Design

The access control flow was designed as a sequential verification chain with no bypass paths:

Step	Action	Outcome on Failure

1	User submits credentials to login.html	Generic error — no user enumeration
2	Credentials validated against hardcoded store (demo)	Denied — no role set
3	Role written to localStorage on success	N/A — only reached on valid auth
4	User redirected to role-specific dashboard	Access Denied — redirect to login
5	Dashboard re-reads and validates role on every load	Access Denied — redirect to login

4. Phase 3 — Implementation

4.1 Authentication Module

The login portal implements credential validation in client-side JavaScript for the demo scope. Credentials are compared against a predefined set (admin/admin123 and employee/emp123). On success, the authenticated role is stored in localStorage and the user is redirected. On failure, a generic alert prevents username enumeration.

4.2 Role Enforcement

Each protected page (admin.html, employee.html) contains a role-check script block in the section, which executes before the page body renders. This ensures that even if a user navigates directly to the URL, they are blocked before any sensitive content is loaded into the DOM.

```
// Admin dashboard role enforcement (head section)
let role = localStorage.getItem('role');
if(role !== 'admin'){
  alert('Access Denied');
  window.location.href = 'index.html';
}
```

4.3 Docker Containerisation

The application is packaged in a minimal Nginx Docker image. The Dockerfile copies the app/ directory into the Nginx web root and exposes port 80. This ensures the application runs in an isolated environment with no dependency on the host operating system.

```
FROM nginx:latest
COPY app/ /usr/share/nginx/html/
EXPOSE 80
```

5. Phase 4 — Security Testing

Seven security test scenarios were executed to validate ZTA enforcement. Each test was designed to simulate a real-world attack vector:

Test ID	Scenario	Method	Expected	Result
T-01	Admin login	Valid credentials	Admin dashboard	PASS
T-02	Employee login	Valid credentials	Employee dashboard	PASS
T-03	Privilege escalation	Employee navigates to admin.html	Access Denied	PASS

T-04	Direct URL bypass	Unauthenticated → admin.html	Redirect to login	PASS
T-05	Invalid credentials	Wrong password	Generic error	PASS
T-06	Brute-force simulation	Repeated wrong attempts	All rejected	PASS
T-07	Session validation	All pages on load	Role checked	PASS

6. Phase 5 — Evaluation

6.1 ZTA Compliance Assessment

ZTA Principle	Implemented	Notes
Never Trust, Always Verify	Yes	Role re-checked on every page load
Least Privilege	Yes	Each role sees only its own dashboard
Micro-Segmentation	Yes	Dashboards are fully isolated
Continuous Monitoring	Partial	Alerts shown; DB logging in roadmap
Minimal Attack Surface	Yes	Single login entry point, no role selector
Assume Breach	Partial	MFA and HTTPS/TLS listed in roadmap

7. Limitations & Roadmap

- Credentials are currently hardcoded in client-side JavaScript — a production system must use server-side authentication with a hashed credential store.
- localStorage is manipulable by a user with browser dev tools — JWT-based server sessions are recommended for production.
- MFA (TOTP) is planned but not yet implemented.
- HTTPS/TLS via Let's Encrypt is in the roadmap for encrypted transport.
- Login attempt logging to the database and a SIEM-style audit dashboard are planned.